Effectful Computation on Propagators: The Layered Recovery Principle

The Prologos Project - from Zachary Larson

2026-03-06

Contents

1 The Fundamental Limit **2**

2 The Insight: Layered Recovery **2**

3 The Precedent: Logic Engine **3**

4 The Application: Session-Derived Effect Ordering **4**
- 4.1 Session Types as Causal Clocks 4
- 4.2 The Galois Connection 4
- 4.3 Cross-Channel Data Dependencies 5
- 4.4 The Five-Layer Architecture 5

5 The `rel~/~proc` Parallel **6**

6 What This Means: Designing for the Principle **7**

7 The Architecture Decision: A + D **7**

8 Relationship to Other Principles **8**

9 Implications for the Language **8**
- 9.1 Effect Ordering Is a Type-System Property 8
- 9.2 Concurrent Effects Fall Out Naturally 9
- 9.3 Static Deadlock Detection 9
- 9.4 The CALM Optimality 9

10 Theoretical Foundations **9**
10.1 CALM Theorem (Hellerstein 2011) 9
10.2 Propagator Networks (Radul & Sussman 2009) 10
10.3 Session Types (Caires & Pfenning 2010, Wadler 2012) 10
10.4 Galois Connections 10
10.5 Vector Clocks (Lamport 1978, Fidge/Mattern 1988) 10
10.6 Partial Order Planning (Weld 1994) 10
10.7 Effect Quantales (Katsumata 2014, Gordon 2021) 10
10.8 Category-Graded Monads (Katsumata, Orchard et al.) 11

11 Inspirations **11**

1 The Fundamental Limit

Propagator networks compute monotone fixed points on lattice-valued cells. Information only grows; values only refine. This is the source of their power — convergence is guaranteed, parallelism is safe, and partial information is first-class.

But IO effects are not monotone. Writing to a file, printing to a console, opening a network connection — these are *irreversible observations* that produce side effects in the world. They cannot be undone, retried, or merged. The CALM theorem (Hellerstein 2011) formalizes this: non-monotonic operations require coordination. You cannot achieve confluence for effectful computation without synchronization.

This creates a tension. Prologos uses propagator networks as its computation substrate. Session types specify protocols that include IO effects. The propagator network must somehow handle effects that violate its fundamental monotonicity assumption.

The question is not whether coordination is needed — it is — but where coordination is placed, and how much of the reasoning about effects can remain monotone.

2 The Insight: Layered Recovery

The solution is not to extend propagators with non-monotone operations. It is to *layer* a control mechanism on top of the monotone substrate, so that:

- **Reasoning about effects** (what effects exist, in what order, with what dependencies) happens monotonically in the propagator network

- **Executing effects** (performing the actual IO) happens at a control boundary, sequentially, after the monotone reasoning reaches a fixed point

This is the **Layered Recovery Principle**:

> Non-monotone behavior is recovered on a monotone substrate by inserting control layers between phases of monotone computation. The control layers perform the non-monotone operations (effect execution, negation evaluation, choice commitment) at precisely the points where coordination is required.

This principle is not specific to IO effects. It is the general mechanism by which Prologos extends the propagator substrate to handle any non-monotone domain.

3 The Precedent: Logic Engine

The first application of Layered Recovery in Prologos is the logic engine (`rel`), which recovers non-monotone logical operations on the propagator substrate:

Layer	Component	Role
1	Propagator Network	Monotone constraint propagation (unification)
2	ATMS	Hypothetical reasoning (choice points)
3	Stratification Controller	Non-monotone operations (negation, cut)

- **Layer 1** handles the monotone core: unification constraints propagate through cells, values refine, the network converges.
- **Layer 2** (ATMS) manages alternative worldviews. Each choice point creates an assumption; the ATMS tracks which combinations of assumptions are consistent (and which are nogoods). This is still monotone — adding nogoods only shrinks the set of valid worldviews.
- **Layer 3** is the stratification barrier. Negation (`\+` in Prolog) is non-monotone: "P is not provable" can become false if P is later derived. Stratification resolves this by dividing the program into strata, computing each stratum to a fixed point before evaluating negation that references lower strata. The barrier between strata is the coordination point.

The key observation: Layers 1 and 2 are entirely monotone and live within the propagator network. Layer 3 is the *minimal* non-monotone extension, placed at exactly the boundary required by the CALM theorem. The logic engine doesn't abandon the propagator model — it *layers* control on top of it.

See: `docs/tracking/2026-02-24_LOGIC_ENGINE_DESIGN.org` §5 (Three-Layer Architecture), Appendix B (Stratification).

4 The Application: Session-Derived Effect Ordering

The second application of Layered Recovery handles IO effects. The insight that unlocks it: *session types are causal clocks*. Each position in a session type's continuation chain is a causally-ordered event. The ordering is not imposed externally — it is **intrinsic** to the session type's structure.

4.1 Session Types as Causal Clocks

A session type `!String . ?Int . end` encodes three positions:

```
Position 0: send a String    (happens first)
Position 1: recv an Int      (happens second)
Position 2: end              (happens last)
```

Session advancement — the monotone process of advancing a session cell from one protocol state to the next — is a *Lamport clock*. Each advancement ticks the clock forward. For multiple channels, the per-channel clocks form a *vector clock*: operations on different channels are concurrent (incomparable in the partial order), while operations on the same channel are totally ordered.

This is the same causal structure that distributed systems use for ordering events, but derived from the *type system* rather than assigned at runtime. Session fidelity (a theorem of the session type system) guarantees that effects respect this ordering.

4.2 The Galois Connection

The formal bridge between sessions and effects is a Galois connection:

```
(alpha, gamma)
```

```
alpha : Session -> EffectPosition      (extract causal position from session state)
gamma : EffectPosition -> Session      (reconstruct remaining protocol from position)
```

Both functions are monotone. The adjunction guarantees soundness: session fidelity implies effect ordering correctness. At a deeper level, both the session lattice and the effect position lattice are *effect quantales* — lattice-ordered monoids with sequential composition — and the Galois connection is a quantale morphism preserving both ordering and composition.

This formalizes why session types can serve as the foundation for effect ordering: the algebraic structure is shared. The connection is not accidental; it reflects a deep mathematical relationship between protocols and effects.

See: `docs/tracking/2026-03-06_SESSION_TYPES_AS_EFFECT_ORDERING.org` §4 (Galois Connection), §6 (Effect Quantale Connection).

4.3 Cross-Channel Data Dependencies

Session types provide per-channel ordering but not cross-channel ordering. When a value received on channel `a` flows to a send on channel `b`, there is a data dependency that the session types alone don't capture.

The resolution: compute the **transitive closure** of the union of session ordering edges and data-flow edges. Transitive closure is a monotone operation on sets of edges — adding an edge never removes existing ordering relationships. This means a propagator can incrementally compute the complete partial order:

```
Session edges:    (a0 < a1), (b0 < b1)           (from session types)
Data-flow edge:   (a0 < b0)                       (x flows from recv a to send b)
Transitive close: (a0 < a1), (b0 < b1), (a0 < b0), (a0 < b1)
```

The transitive closure is a monotone fixed point. It lives natively in the propagator network. No external scheduling is needed.

4.4 The Five-Layer Architecture

The full architecture for effectful computation on propagators:

```
Layer 1: Session Advancement              [exists -- session-runtime.rkt]
    Session cells advance monotonically through session types.
    Each advancement is a causal clock tick for that channel.

Layer 2: Data-Flow Analysis               [NEW]
```

```
    Analyze variable bindings to derive cross-channel ordering edges.
    recv x a -> send [f x] b  ==>  ordering edge (a_pos < b_pos)

Layer 3: Transitive Closure                [NEW]
    Compute the complete partial order by propagating ordering edges
    to their transitive closure. Monotone fixed-point computation.
    Concurrent effects remain unordered (least commitment).

Layer 4: ATMS Branching                    [exists -- atms.rkt]
    Maintain per-branch ordering hypotheses for proc-case.
    Resolve when branch is chosen; discard inconsistent worldviews.

Layer 5: Effect Handler                    [NEW]
    Read the resolved partial order. Execute effects in any valid
    linearization. This is the only non-monotone step (actual IO).
```

Layers 1–4 are monotone. Layer 5 is the control boundary — the precise point where the CALM theorem requires coordination.

5 The `rel~`/`~proc` Parallel

The pattern reveals a deep structural parallel between Prologos's two computational engines:

Dimension	`rel` (Logic Engine)	`proc` (Session Runtime)
Variables	Logic variables (unification cells)	Session cells (protocol state)
Constraints	Equality constraints (type/term)	Ordering constraints (causal)
Propagation	Unification propagators	Session advancement + ordering propagators
Fixed point	Constraint closure (all equalities)	Ordering closure (transitive causal edges)
Hypotheses	ATMS worldviews (choice points)	ATMS worldviews (branch alternatives)
Resolution	Evidence selects worldview	Received value selects worldview
Execution	Proof terms (constructive evidence)	Effects (IO operations in causal order)
Control layer	Stratification (negation evaluation)	Effect handler (IO execution)

Both engines share the three-layer architecture: Propagator Network (monotone fixed points) + ATMS (hypothetical reasoning) + Control Layer (non-monotone operations). The Layered Recovery Principle is the *common theory* underlying both.

This parallel is not coincidental. Both `rel` and `proc` face the same fundamental challenge: how to perform non-monotone operations (negation for

`rel`; effects for `proc`) on a monotone substrate (propagator network). Both solve it the same way: reason monotonically about the non-monotone domain, then execute the non-monotone operations at a control barrier after the monotone reasoning converges.

6 What This Means: Designing for the Principle

The Layered Recovery Principle is a design methodology, not just an observation. When extending Prologos to handle a new domain that involves non-monotone operations:

1. **Identify the monotone core**: What can be reasoned about convergently? For effects, it's ordering relationships. For logic, it's unification constraints. For type inference, it's subtyping relationships.
2. **Encode the monotone core in the propagator network**: Define lattices, cells, and propagators for the convergent reasoning. This gets parallelism, incrementality, and partial-information handling for free.
3. **Identify the non-monotone boundary**: What operations require coordination? For effects, it's execution. For logic, it's negation. This is where the CALM theorem demands synchronization.
4. **Insert a control layer at the boundary**: The control layer reads the fixed point of the monotone core and performs the non-monotone operations in a coordinated fashion.
5. **Use the ATMS for hypothetical reasoning**: If the domain involves alternatives (choice points, branches, speculative evaluation), the ATMS manages the hypotheses monotonically.

This methodology has now been applied successfully to two domains (logic, effects) and the consistent results suggest it will generalize to others.

7 The Architecture Decision: A + D

Two architectures for effect ordering:

- **Architecture A (Stratified Barriers)**: Effects execute in AST walk order. Sequential, simple, correct by construction. No propagator involvement in ordering. This is the current implementation for `main` and top-level REPL expressions.

- **Architecture D (Session-Derived Ordering)**: Effects are ordered by session type structure via the Galois connection. Cross-channel dependencies resolved by transitive closure. Branches handled by ATMS. The ordering is a theorem from session fidelity, not a design choice.

The recommended architecture is **A** + **D**:

- **D** for session-typed IO (the structured, common case). Ordering is derived from session types, verified by the type checker, and compositional across protocol phases. Concurrent effects on independent channels are correctly unordered (least commitment).
- **A** for unsessioned IO (the REPL, top-level eval, `main` without explicit sessions). Walk-based ordering is simple and sufficient.

Architecture A is, in fact, a **degenerate case** of Architecture D: `main` with sequential IO and no explicit session is equivalent to a process with a single implicit session. As `main` gains explicit session structure, D's richer ordering semantics apply automatically.

Architecture B (Timestamped Effect Cells) is **subsumed**: for session-typed effects, D provides strictly more information (intrinsic, verified ordering); for unsessioned effects, A is simpler. B adds nothing that A + D don't provide.

8 Relationship to Other Principles

Document	Relationship
`DESIGN_PRINCIPLES.org`	"Correctness Through Types" — effect ordering is derived from
`PROTOCOLS_AS_TYPES.org`	Session type composition (protocol libraries) composes effect or
`LANGUAGE_VISION.org`	"Protocol correctness" extends to effect correctness. "Session t
`CAPABILITY_SECURITY.md`	`SysCap` as powerbox for `main` (Architecture A fallback). Capab
`RELATIONAL_LANGUAGE_VISION.org`	The logic engine (`rel`) is the first application of Layered Recov
IO Implementation PIR	§4a ("Propagators Cannot Order Side Effects") is the discovery

9 Implications for the Language

9.1 Effect Ordering Is a Type-System Property

With Architecture D, the ordering of IO effects is not a scheduler decision or a compiler implementation detail. It is a **property of the type system**.

The session type determines the ordering; the type checker verifies it; the compiler derives it via the Galois connection. Users can reason about effect ordering by reading the session type — the same way they reason about protocol correctness.

9.2 Concurrent Effects Fall Out Naturally

For multi-channel processes, effects on independent channels are concurrent — the partial order leaves them unordered. This means the runtime can execute them in parallel or in any interleaving, without violating correctness. The session type system guarantees that these effects are on independent resources (different channels, verified by linearity). Concurrency is not opt-in; it's the default for independent effects.

9.3 Static Deadlock Detection

The transitive closure computation for cross-channel data dependencies can detect cycles (`A < B < A`), which indicate deadlocks. This turns runtime deadlocks into compile-time errors. The ordering analysis serves double duty: it provides effect ordering *and* deadlock detection for data-dependent cross-channel interactions.

9.4 The CALM Optimality

The architecture achieves the minimum coordination required by the CALM theorem. All reasoning about effects (ordering, dependencies, alternatives) is monotone and coordination-free. The only coordination point is the final effect execution barrier (Layer 5). This is provably optimal — no architecture can eliminate this barrier without violating the CALM theorem.

10 Theoretical Foundations

10.1 CALM Theorem (Hellerstein 2011)

The Consistency And Logical Monotonicity theorem: a distributed program has a consistent, coordination-free implementation if and only if it is monotonic. IO effects are non-monotonic, so coordination is required. The Layered Recovery Principle minimizes this coordination to a single barrier.

10.2 Propagator Networks (Radul & Sussman 2009)

Monotone fixed-point computation on lattice-valued cells. The computational substrate for Prologos. Propagator designers explicitly excluded non-commutative effects from the model — the Layered Recovery Principle is our answer to this exclusion.

10.3 Session Types (Caires & Pfenning 2010, Wadler 2012)

Session types as linear logic propositions via Curry-Howard. The continuation structure of a session type encodes the causal ordering of communication events. Session fidelity guarantees that implementations respect this ordering.

10.4 Galois Connections

An adjunction `(\alpha, \gamma)` between ordered domains. Used throughout abstract interpretation (Cousot & Cousot) for soundly approximating one domain in terms of another. Our use connects the session lattice to the effect position lattice.

10.5 Vector Clocks (Lamport 1978, Fidge/Mattern 1988)

Per-process Lamport clocks composed into vectors for partial ordering of events in distributed systems. Per-channel session depths composed into vectors for partial ordering of effects across channels. The mathematical structure is identical.

10.6 Partial Order Planning (Weld 1994)

Least-commitment planning with causal links and ordering constraints. Effect ordering via transitive closure is isomorphic to POP: data-flow edges are causal links, session ordering is initial constraints, `proc-case` is plan alternatives, ATMS is threat postponement.

10.7 Effect Quantales (Katsumata 2014, Gordon 2021)

Lattice-ordered monoids with sequential composition that distribute over joins. Both session types and effect positions form effect quantales. The Galois connection is a quantale morphism preserving this algebraic structure.

10.8 Category-Graded Monads (Katsumata, Orchard et al.)

Unifies graded monads (effect systems) and parameterised monads (session types) into a single categorical framework. The Galois connection between sessions and effects is, categorically, a functor between indexing categories.

11 Inspirations

- **Hellerstein** (CALM theorem) — The theoretical foundation for why coordination is necessary and where it must be placed.
- **Radul & Sussman** (Propagators) — The computational substrate. Their explicit exclusion of non-commutative effects is the challenge we resolve.
- **Caires & Pfenning** (Session types as linear logic) — The Curry-Howard foundation that makes session types causal clocks.
- **Lamport** (Logical clocks) — Session advancement IS a Lamport clock. The connection is direct, not analogical.
- **de Kleer** (ATMS) — Hypothetical reasoning for branching alternatives. The same ATMS serves both `rel` and `proc`.
- **Weld** (Partial Order Planning) — The least-commitment principle for ordering: don't impose unnecessary ordering on concurrent effects.
- **Atkey** (Parameterised monads) — Session types as pre/post state indices. Sequential composition = bind matching postcondition to precondition.
- **The Prologos Logic Engine** — The first application of Layered Recovery, which established the three-layer architecture and demonstrated that non-monotone operations can be recovered on a monotone substrate.